KOÇ UNIVERSITY

COMP 350
Introduction to DevOps

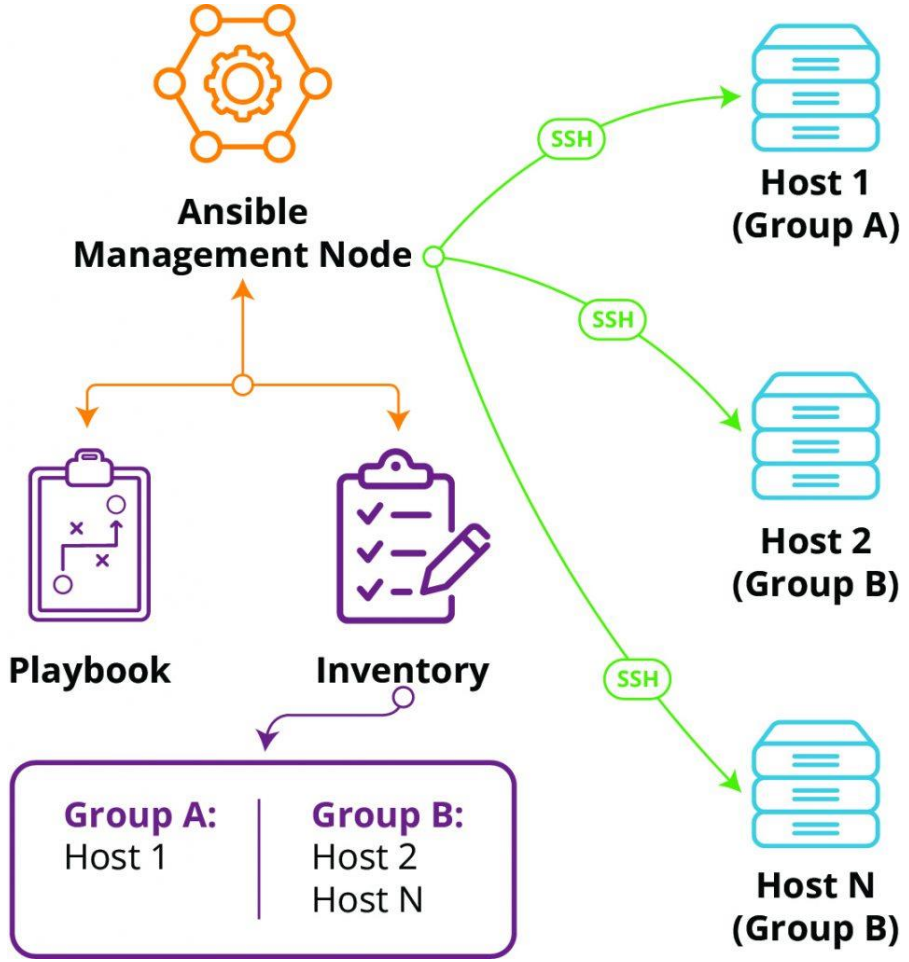
Lecture 13
Ansible (Part 2)

Hakan Ayral

Ansible Modules

- We mentioned three main files that you need to consider:
  1. **Inventory** file: Contains the entry of the nodes that need to be managed
  2. **Ansible.cfg** file: Located by default at /etc/ansible/ansible.cfg, it has the necessary privilege escalation options and the location of the inventory file. This is the global config file.
  3. **Main** file: A **playbook** that has modules that perform various tasks on a host listed in an inventory or host file

- Ansible **module**s are python scripts that can be used inside an Ansible **playbook**.
  - A **playbook** consists of a **play.**
  - A **play** consists of **tasks.**



Installing Debian packages with Ansible

- Ansible has the builtin module ansible.builtin.apt which manages apt packages (such as for Debian/Ubuntu)
- Usage examples:

```yaml
- name: Install apache httpd  (state=present is optional)
  ansible.builtin.apt:
    name: apache2
    state: present

- name: Update repositories cache and install "foo" package
  ansible.builtin.apt:
    name: foo
    update_cache: yes

- name: Remove "foo" package
  ansible.builtin.apt:
    name: foo
    state: absent

- name: Install the package "foo"
  ansible.builtin.apt:
    name: foo
```

```yaml
- name: Install a list of packages
  ansible.builtin.apt:
    pkg:
    - foo
    - foo-tools

- name: Install the version '1.00' of package "foo"
  ansible.builtin.apt:
    name: foo=1.00

- name: Update the repository cache and update package "nginx" to latest
  ansible.builtin.apt:
    name: nginx
    state: latest
    default_release: squeeze-backports
    update_cache: yes
```

Controlling Background Services

- After installing a package, you may need to start a service.
  - Especially if the package you installed is a server.
- The ansible.builtin.service module enables you to start, stop, and reload installed services.
- Usage examples:

```yaml
- name: Start service httpd, if not started
  ansible.builtin.service:
    name: httpd
    state: started

- name: Stop service httpd, if started
  ansible.builtin.service:
    name: httpd
    state: stopped

- name: Restart service httpd, in all cases
  ansible.builtin.service:
    name: httpd
    state: restarted

- name: Reload service httpd, in all cases
  ansible.builtin.service:
    name: httpd
    state: reloaded
```

```yaml
- name: Enable service httpd, and not touch the state
  ansible.builtin.service:
    name: httpd
    enabled: yes

- name: Start service foo, based on running process /usr/bin/foo
  ansible.builtin.service:
    name: foo
    pattern: /usr/bin/foo
    state: started

- name: Restart network service for interface eth0
  ansible.builtin.service:
    name: network
    state: restarted
    args: eth0
```

Copy files to remote locations

- The ansible.builtin.copy module copies a file or a directory structure from the local or remote machine to a location on the remote machine.
- Usage examples:

```yaml
- name: Copy file with owner and permissions
  ansible.builtin.copy:
    src: /srv/myfiles/foo.conf
    dest: /etc/foo.conf
    owner: foo
    group: foo
    mode: '0644'

- name: Copy file with owner and permission, using symbolic representation
  ansible.builtin.copy:
    src: /srv/myfiles/foo.conf
    dest: /etc/foo.conf
    owner: foo
    group: foo
    mode: u=rw,g=r,o=r

- name: Another symbolic mode example, adding some permissions and removing others
  ansible.builtin.copy:
    src: /srv/myfiles/foo.conf
    dest: /etc/foo.conf
    owner: foo
    group: foo
    mode: u+rw,g-wx,o-rwx
```

```yaml
- name: Copy a new "ntp.conf" file into place, backing up the original if it differs
  ansible.builtin.copy:
    src: /mine/ntp.conf
    dest: /etc/ntp.conf
    owner: root
    group: root
    mode: '0644'
    backup: yes

- name: Copy a new "sudoers" file into place, after passing validation with visudo
  ansible.builtin.copy:
    src: /mine/sudoers
    dest: /etc/sudoers
    validate: /usr/sbin/visudo -csf %s

- name: Copy a "sudoers" file on the remote machine for editing
  ansible.builtin.copy:
    src: /etc/sudoers
    dest: /etc/sudoers.edit
    remote_src: yes
    validate: /usr/sbin/visudo -csf %s
```

Manage files and file properties

- ansible.builtin.file module let's you set attributes of files, directories, or symlinks and their targets. Also it can remove files, symlinks or directories.
- Usage examples:

```yaml
- name: Change file ownership, group and permissions
  ansible.builtin.file:
    path: /etc/foo.conf
    owner: foo
    group: foo
    mode: '0644'

- name: Give insecure permissions to an existing file
  ansible.builtin.file:
    path: /work
    owner: root
    group: root
    mode: '1777'

- name: Create a symbolic link
  ansible.builtin.file:
    src: /file/to/link/to
    dest: /path/to/symlink
    owner: foo
    group: foo
    state: link
```

```yaml
- name: Create two hard links
  ansible.builtin.file:
    src: '/tmp/{{ item.src }}'
    dest: '{{ item.dest }}'
    state: hard
  loop:
    - { src: x, dest: y }
    - { src: z, dest: k }

- name: Touch a file, using symbolic modes to set the permissions (equivalent to 0644)
  ansible.builtin.file:
    path: /etc/foo.conf
    state: touch
    mode: u=rw,g=r,o=r

- name: Touch the same file, but add/remove some permissions
  ansible.builtin.file:
    path: /etc/foo.conf
    state: touch
    mode: u+rw,g-wx,o-rwx
```

Manage lines in text files

- ansible.builtin.lineinfile module ensures a particular line is in a file, or replace an existing line using a regular expression. This is useful if you want to change a single line in a file only.
- Usage examples:

```yaml
- name: Ensure SELinux is set to enforcing mode
  ansible.builtin.lineinfile:
    path: /etc/selinux/config
    regexp: '^SELINUX='
    line: SELINUX=enforcing

- name: Make sure group wheel is not in the sudoers configuration
  ansible.builtin.lineinfile:
    path: /etc/sudoers
    state: absent
    regexp: '^%wheel'

- name: Replace a localhost entry with our own
  ansible.builtin.lineinfile:
    path: /etc/hosts
    regexp: '^127\.0\.0\.1'
    line: 127.0.0.1 localhost
    owner: root
    group: root
    mode: '0644'
```

```yaml
- name: Replace a localhost entry searching for a literal string to avoid escaping
  ansible.builtin.lineinfile:
    path: /etc/hosts
    search_string: '127.0.0.1'
    line: 127.0.0.1 localhost
    owner: root
    group: root
    mode: '0644'

- name: Ensure the default Apache port is 8080
  ansible.builtin.lineinfile:
    path: /etc/httpd/conf/httpd.conf
    regexp: '^Listen '
    insertafter: '^#Listen '
    line: Listen 8080

- name: Ensure php extension matches new pattern
  ansible.builtin.lineinfile:
    path: /etc/httpd/conf/httpd.conf
    search_string: '<FilesMatch ".php[45]?$">'
    insertafter: '^\t<Location \/>\n'
    line: '        <FilesMatch ".php[34]?$">'
```

Deploy software (or files) from git

- ansible.builtin.git module manages git checkouts of repositories to deploy files or software.
- Usage examples:

```yaml
- name: Git checkout
  ansible.builtin.git:
    repo: 'https://foosball.example.org/path/to/repo.git'
    dest: /srv/checkout
    version: release-0.22

- name: Read-write git checkout from github
  ansible.builtin.git:
    repo: git@github.com:mylogin/hello.git
    dest: /home/mylogin/hello

- name: Just ensuring the repo checkout exists
  ansible.builtin.git:
    repo: 'https://foosball.example.org/path/to/repo.git'
    dest: /srv/checkout
    update: no
```

```yaml
- name: Just get information about the repository whether or not it has already been cloned locally
  ansible.builtin.git:
    repo: 'https://foosball.example.org/path/to/repo.git'
    dest: /srv/checkout
    clone: no
    update: no

- name: Checkout a github repo and use refspec to fetch all pull requests
  ansible.builtin.git:
    repo: https://github.com/ansible/ansible-examples.git
    dest: /src/ansible-examples
    refspec: '+refs/pull/*:refs/heads/*'

- name: Create git archive from repo
  ansible.builtin.git:
    repo: https://github.com/ansible/ansible-examples.git
    dest: /src/ansible-examples
    archive: /tmp/ansible-examples.zip
```

Create a compressed archive of one or more files

- community.general.archive module creates or extends an archive.
- The source and archive are on the remote host, and the archive is not copied to the local host.
  - Source files can be deleted after archival by specifying remove=True.
- Usage example:

```yaml
- name: Compress directory /path/to/foo/ into /path/to/foo.tgz
  community.general.archive:
    path: /path/to/foo
    dest: /path/to/foo.tgz

- name: Compress regular file /path/to/foo into /path/to/foo.gz and remove it
  community.general.archive:
    path: /path/to/foo
    remove: true

- name: Create a zip archive of /path/to/foo
  community.general.archive:
    path: /path/to/foo
    format: zip
```

```yaml
- name: Create a bz2 archive of multiple files, rooted at /path
  community.general.archive:
    path:
    - /path/to/foo
    - /path/wong/foo
    dest: /path/file.tar.bz2
    format: bz2

- name: Create a bz2 archive of a globbed path, while excluding specific dirnames
  community.general.archive:
    path:
    - /path/to/foo/*
    dest: /path/file.tar.bz2
    exclude_path:
    - /path/to/foo/bar
    - /path/to/foo/baz
    format: bz2
```

Execute commands on targets

- The ansible.builtin.command module takes the command name followed by a list of space-delimited arguments.
  - The command(s) will not be processed through the shell, so operations like "*", "<", ">", "|", ";" and "&" will not work.
  - Also, environment variables are resolved via Python, not shell.
- Usage examples:

```yaml
- name: Return motd to registered var
  ansible.builtin.command: cat /etc/motd
  register: mymotd

# free-form (string) arguments, all arguments on one line
- name: Run command if /path/to/database does not exist (without 'args')
  ansible.builtin.command: /usr/bin/make_database.sh db_user db_name creates=/path/to/database

# free-form (string) arguments, some arguments on separate lines with the 'args' keyword
# 'args' is a task keyword, passed at the same level as the module
- name: Run command if /path/to/database does not exist (with 'args' keyword)
  ansible.builtin.command: /usr/bin/make_database.sh db_user db_name
  args:
    creates: /path/to/database

# 'cmd' is module parameter
- name: Run command if /path/to/database does not exist (with 'cmd' parameter)
  ansible.builtin.command:
    cmd: /usr/bin/make_database.sh db_user db_name
    creates: /path/to/database
```

Ansible playbooks

- Ansible Playbooks offer a repeatable, reusable, simple configuration management and multi-machine deployment system, well suited to deploying complex applications.

- If you need to execute a task with Ansible **more than once**, write a playbook; then you can use the playbook to push out new configuration or confirm the configuration of remote systems.

- Playbooks can:
  - declare configurations
  - orchestrate steps of any manual ordered process, on multiple sets of machines, in a defined order
  - launch tasks synchronously or asynchronously

Playbook syntax

- Playbooks are expressed in YAML format with a minimum of syntax.
- A **playbook** is composed of one or more '**plays**' in an ordered list.
  - Each **play** executes part of the overall goal of the **playbook**, running one or more **tasks**.
  - Each **task** calls an Ansible **module**.

- A **playbook** runs in order from top to bottom.
  - Within each **play**, **tasks** also run in order from top to bottom.
  - **Playbooks** with multiple '**plays**' can orchestrate multi-machine deployments, running one **play** on your webservers, then another **play** on your database servers, then a third **play** on your network infrastructure, and so on.

- Each play defines at least two things:
  1. the managed nodes to target, using a pattern
  2. at least one task to execute

To run a playbook, use the ansible-playbook command:

```
ansible-playbook playbook.yml -f 10
```